BỘ THÔNG TIN VÀ TRUYỀN THÔNG
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN

Báo cáo đồ án môn học — LAB 01

BÁO CÁO ĐỒ ÁN MÔN HỌC

XÂY DỰNG CƠ SỞ DỮ LIỆU QUAN HỆ VÀ KHAI THÁC DỮ LIỆU BÁN HÀNG TRÊN SQLITE VỚI GOOGLE CLOUD SHELL

Giảng viên hướng dẫn: .....
Sinh viên thực hiện : Nguyễn Danh Thái
Mã sinh viên : B23DCKH105
Lớp : .....
Ngành : Công nghệ thông tin

Introduction to Data Science

HÀ NỘI, THÁNG 11 NĂM 2026

TÓM TẮT ĐỒ ÁN

Đồ án trình bày quá trình xây dựng và khai thác một cơ sở dữ liệu quan hệ cho bài toán bán hàng trực tuyến, thực hiện hoàn toàn trên nền tảng miễn phí: Google Cloud Shell cung cấp môi trường dòng lệnh, Python cung cấp thư viện sqlite3, và SQLite đóng vai trò hệ quản trị cơ sở dữ liệu nhúng.

Cơ sở dữ liệu gồm bốn bảng customers, products, orders và order_items với khoá chính, khoá ngoại và các ràng buộc CHECK. Đồ án cài đặt đầy đủ 21 yêu cầu của đề bài: nhóm truy vấn cơ bản và thao tác dữ liệu (S01–S07), nhóm truy vấn tổng hợp nhiều bảng (S08–S13), nhóm chỉ mục và phân tích kế hoạch thực thi (S14–S15), và nhóm mở rộng (S16–S21) trên bộ dữ liệu sinh tự động gồm 100 khách hàng, 30 sản phẩm, 500 đơn hàng và 1.500 dòng đơn hàng.

Kết quả thực nghiệm cho thấy toàn bộ câu lệnh thực thi không lỗi và có thể chạy lặp an toàn; kế hoạch thực thi của truy vấn lọc theo khách hàng chuyển từ SCAN (đọc tuần tự toàn bảng) sang SEARCH dùng chỉ mục, ở dạng covering index. Đồ án cũng chỉ ra giới hạn của việc đánh giá chỉ mục bằng thời gian chạy trên dữ liệu nhỏ và đề xuất cách đọc EXPLAIN QUERY PLAN thay thế.

Từ khoá: cơ sở dữ liệu quan hệ, SQLite, SQL, khoá ngoại, chỉ mục, EXPLAIN QUERY PLAN, Google Cloud Shell.

MỤC LỤC

TÓM TẮT ĐỒ ÁN **2**
DANH MỤC TỪ VIẾT TẮT **5**
DANH MỤC HÌNH VẼ **6**
DANH MỤC BẢNG BIỂU **7**
CHƯƠNG 1. Mở đầu **8**
1.1. Lý do chọn đề tài 8
1.2. Mục tiêu của đồ án 8
1.3. Đối tượng và phạm vi nghiên cứu 9
1.4. Phương pháp thực hiện 9
1.5. Bố cục báo cáo 9
CHƯƠNG 2. Cơ sở lý thuyết **10**
2.1. Cơ sở dữ liệu quan hệ 10
2.2. Hệ quản trị cơ sở dữ liệu SQLite 10
2.3. Ngôn ngữ SQL và các kỹ thuật truy vấn 11
2.4. Chỉ mục và kế hoạch thực thi 12
2.5. Môi trường thực hành: Google Cloud Shell 12
CHƯƠNG 3. Phân tích và thiết kế **13**
3.1. Yêu cầu của bài toán 13
3.2. Thiết kế mô hình dữ liệu 13
3.3. Thiết kế kiến trúc chương trình 15
3.4. Thiết kế dữ liệu kiểm thử 15
CHƯƠNG 4. Triển khai và thực nghiệm **17**
4.1. Môi trường thực hành 17
4.2. Khởi tạo cơ sở dữ liệu 17
4.3. Cài đặt nhóm truy vấn cơ bản và CRUD (S01–S07) 18
4.4. Cài đặt truy vấn tổng hợp nhiều bảng (S08–S13) 19
4.5. Chỉ mục và phân tích kế hoạch thực thi (S14–S15) 20
4.6. Sinh dữ liệu mở rộng và các yêu cầu S16–S21 21
4.7. Kiểm thử 24
CHƯƠNG 5. Kết quả và đánh giá **25**
5.1. Kết quả đạt được theo tiêu chí 25
5.2. Đánh giá hiệu quả của chỉ mục 25
5.3. Đánh giá tính đúng đắn và khả năng tái lập 25
5.4. Hạn chế 26
5.5. Bài học kinh nghiệm 26
CHƯƠNG 6. Kết luận và hướng phát triển **27**
6.1. Kết luận 27
6.2. Hướng phát triển 27
TÀI LIỆU THAM KHẢO **28**
CHƯƠNG A. Cấu trúc bài nộp và hướng dẫn tái lập **29**
CHƯƠNG B. Mã nguồn chính **30**
B.1. Lược đồ cơ sở dữ liệu (schema.sql) 30
B.2. Đáp án nhóm bắt buộc (answers.sql, S01–S15) 30
B.3. Đáp án nhóm mở rộng (answers_homework.sql, S16–S21) 32
CHƯƠNG C. Kết quả đo và ảnh minh chứng **35**

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Diễn giải
CSDL	Cơ sở dữ liệu
HQTCSDL	Hệ quản trị cơ sở dữ liệu
DDL	Data Definition Language — ngôn ngữ định nghĩa dữ liệu
DML	Data Manipulation Language — ngôn ngữ thao tác dữ liệu
DQL	Data Query Language — ngôn ngữ truy vấn dữ liệu
PK	Primary Key — khoá chính
FK	Foreign Key — khoá ngoại
SQL	Structured Query Language — ngôn ngữ truy vấn có cấu trúc
EQP	EXPLAIN QUERY PLAN — lệnh xem kế hoạch thực thi của SQLite
CLI	Command Line Interface — giao diện dòng lệnh
B-tree	Cấu trúc cây cân bằng dùng để lưu chỉ mục
CSDLQT	Cơ sở dữ liệu quan hệ

Ngoài các từ viết tắt trên, báo cáo giữ nguyên tên bảng, tên cột và tên câu lệnh theo đúng quy ước của đề bài (ví dụ customers, customer_id, S01…S21).

DANH MỤC HÌNH VẼ

Hình 3.1. Sơ đồ quan hệ của cơ sở dữ liệu bán hàng 14
Hình 4.1. Kết quả khởi tạo: số dòng bốn bảng và kiểm tra khoá ngoại 18
Hình 4.2. Kết quả truy vấn S08 và S09 trên dữ liệu mẫu 20
Hình 4.3. Kế hoạch thực thi trước và sau khi tạo chỉ mục trên cột customer_id 21
Hình 4.4. Kết quả sinh dữ liệu mở rộng và kiểm tra tính toàn vẹn 22
Hình 4.5. Kết quả xếp hạng khách hàng theo doanh thu với RANK và DENSE_RANK 23
Hình C.1. Môi trường thực hành: phiên bản Python và SQLite 35
Hình C.2. Console SQLite: câu lệnh nhiều dòng, xử lý lỗi và thoát 35
Hình C.3. Số dòng bốn bảng sau khi nạp dữ liệu mẫu 35
Hình C.4. Sinh dữ liệu mở rộng và kiểm tra khoá ngoại 36
Hình C.5. Kết quả S08 và S09 36
Hình C.6. Kế hoạch thực thi trước và sau khi tạo chỉ mục 36
Hình C.7. Kết quả xếp hạng khách hàng với RANK và DENSE_RANK 37

DANH MỤC BẢNG BIỂU

Bảng 1.1. Bố cục của báo cáo 9
Bảng 3.1. Danh sách 21 yêu cầu của đề bài 13
Bảng 3.2. Chi tiết các bảng trong cơ sở dữ liệu 14
Bảng 3.3. Kiến trúc các tệp mã nguồn 15
Bảng 3.4. Quy mô hai bộ dữ liệu kiểm thử 16
Bảng 4.1. Môi trường thực hành 17
Bảng 4.2. Kết quả S08 và S09 trên dữ liệu mẫu 19
Bảng 4.3. Kế hoạch thực thi trước và sau khi tạo chỉ mục 21
Bảng 4.4. Doanh thu theo tháng trên dữ liệu mở rộng (S16) 22
Bảng 4.5. So sánh bảng tạm giữa SQL Server và SQLite (S21) 24
Bảng 4.6. Các kiểm thử đã thực hiện 24
Bảng 5.1. Đối chiếu kết quả với tiêu chí chấm điểm của đề bài 25
Bảng 5.2. So sánh các kiểu truy cập dữ liệu trong SQLite 25
Bảng A.1. Cấu trúc gói bài nộp 29

CHƯƠNG 1. MỞ ĐẦU

1.1. Lý do chọn đề tài

Dữ liệu là trung tâm của mọi hệ thống thông tin hiện đại. Với một cửa hàng trực tuyến, việc trả lời được các câu hỏi như "đơn hàng nào có giá trị cao nhất", "khách hàng nào chi tiêu nhiều nhất", "sản phẩm nào bán chạy" hay "doanh thu từng tháng thay đổi ra sao" đòi hỏi dữ liệu phải được tổ chức theo một mô hình chặt chẽ và truy vấn bằng ngôn ngữ phù hợp. SQL là ngôn ngữ chuẩn cho công việc này, còn mô hình quan hệ là nền tảng lý thuyết đứng sau nó.

Trong điều kiện học tập, sinh viên thường gặp trở ngại về môi trường: cài đặt hệ quản trị như SQL Server hay MySQL tốn tài nguyên, đôi khi phát sinh chi phí hoặc yêu cầu cấu hình phức tạp. Đề bài LAB 01 giải quyết trở ngại đó bằng một lựa chọn gọn nhẹ: dùng SQLite — hệ quản trị nhúng nằm trong thư viện chuẩn của Python — và Google Cloud Shell làm môi trường thực hành miễn phí, chỉ cần tài khoản Gmail và trình duyệt.

Vì vậy đồ án chọn đề tài xây dựng cơ sở dữ liệu quan hệ và khai thác dữ liệu bán hàng trên SQLite, nhằm rèn luyện đồng thời ba nhóm kỹ năng: thiết kế lược đồ có ràng buộc, viết truy vấn từ cơ bản đến nâng cao, và đánh giá hiệu năng truy vấn thông qua chỉ mục cùng kế hoạch thực thi.

1.2. Mục tiêu của đồ án

Đồ án đặt ra các mục tiêu cụ thể sau:

1. Thiết kế và tạo lập được cơ sở dữ liệu bán hàng gồm bốn bảng với khoá chính, khoá ngoại và ràng buộc miền giá trị đúng theo yêu cầu của đề bài.
2. Thực hiện thành thạo các thao tác thêm, đọc, sửa, xoá (CRUD) và các truy vấn nền tảng SELECT, WHERE, ORDER BY.
3. Viết được các truy vấn tổng hợp nhiều bảng: JOIN, GROUP BY, HAVING, truy vấn con và hàm cửa sổ (window function).
4. Tạo chỉ mục và đọc được kế hoạch thực thi do EXPLAIN QUERY PLAN trả về, từ đó giải thích được ảnh hưởng của chỉ mục tới truy vấn.
5. Sinh được bộ dữ liệu kiểm thử quy mô lớn hơn (100 khách hàng, 30 sản phẩm, 500 đơn hàng, 1.000 dòng đơn hàng trở lên) và đánh giá lại kế hoạch thực thi trên bộ dữ liệu đó.

1.3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu là mô hình dữ liệu quan hệ của một cửa hàng trực tuyến giả định, cùng các kỹ thuật truy vấn và tối ưu truy vấn trên SQLite. Phạm vi của đồ án được giới hạn như sau:

- Về dữ liệu: chỉ gồm bốn bảng customers, products, orders, order_items với dữ liệu giả lập, không sử dụng dữ liệu cá nhân thật.
- Về hệ quản trị: SQLite phiên bản trong thư viện chuẩn Python, không dùng máy chủ cơ sở dữ liệu riêng.
- Về môi trường: Google Cloud Shell và máy cá nhân, không bật dịch vụ tính phí.
- Về nội dung: 21 yêu cầu S01–S21 của đề bài; không mở rộng sang giao diện web hay báo cáo trực quan hoá.

1.4. Phương pháp thực hiện

Đồ án được thực hiện theo trình tự sau: (i) nghiên cứu lý thuyết về mô hình quan hệ, ngôn ngữ SQL và chỉ mục; (ii) phân tích yêu cầu của đề, thiết kế lược đồ và kiến trúc các tệp mã nguồn; (iii) cài đặt lần lượt theo từng nhóm yêu cầu, sau mỗi nhóm đều chạy kiểm thử; (iv) đo và phân tích kế hoạch thực thi trước và sau khi tạo chỉ mục; (v) sinh dữ liệu mở rộng, đánh giá lại và viết báo cáo.

Một yêu cầu kỹ thuật được đặt ra xuyên suốt quá trình cài đặt: mọi tệp kịch bản phải chạy lặp được. Cụ thể, khi chạy lại toàn bộ đáp án, dữ liệu phải trở về trạng thái ban đầu thay vì bị cộng dồn hay sinh lỗi trùng khoá. Yêu cầu này được kiểm chứng bằng cách chạy mỗi tệp hai lần liên tiếp và so sánh kết quả.

1.5. Bố cục báo cáo

Báo cáo gồm sáu chương và ba phụ lục, được trình bày trong Bảng 1.1.

Bảng 1.1. Bố cục của báo cáo

Chương	Nội dung chính
1. Mở đầu	Lý do chọn đề tài, mục tiêu, phạm vi và phương pháp thực hiện
2. Cơ sở lý thuyết	Mô hình quan hệ, SQL, chỉ mục B-tree và công cụ kế hoạch thực thi
3. Phân tích và thiết kế	Yêu cầu, lược đồ dữ liệu, kiến trúc chương trình, dữ liệu kiểm thử
4. Triển khai và thực nghiệm	Cài đặt schema, các nhóm truy vấn, chỉ mục, dữ liệu mở rộng, kiểm thử
5. Kết quả và đánh giá	Đối chiếu tiêu chí, đánh giá hiệu quả chỉ mục, hạn chế và bài học
6. Kết luận	Tổng kết kết quả và hướng phát triển
Phụ lục A–C	Cấu trúc bài nộp, mã nguồn chính, kết quả đo và ảnh minh chứng

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Cơ sở dữ liệu quan hệ

2.1.1. Khái niệm cơ bản

Mô hình quan hệ do E. F. Codd đề xuất năm 1970 tổ chức dữ liệu thành các bảng (quan hệ). Mỗi bảng gồm các cột (thuộc tính) và các hàng (bộ). Mỗi ô là một giá trị nguyên tử; quan hệ giữa các bảng được biểu diễn bằng giá trị khoá chứ không bằng con trỏ vật lý. Cách tổ chức này cho phép truy vấn dữ liệu bằng đại số quan hệ và ngôn ngữ khai báo SQL.

2.1.2. Khoá và ràng buộc toàn vẹn

Khoá chính (primary key) là một cột hoặc tổ hợp cột xác định duy nhất mỗi hàng, đồng thời không được nhận giá trị NULL. Khoá ngoại (foreign key) là cột tham chiếu tới khoá chính của bảng khác, dùng để bảo đảm tính toàn vẹn tham chiếu: không thể tồn tại đơn hàng của một khách hàng không có trong bảng khách hàng. Ngoài ra ràng buộc CHECK giới hạn miền giá trị của cột, còn NOT NULL bắt buộc phải có giá trị.

Trong SQLite, khoá ngoại chỉ được kiểm tra khi bật cờ PRAGMA foreign_keys = ON. Đây là khác biệt quan trọng so với nhiều hệ quản trị khác: nếu lập trình viên quên bật, khai báo FOREIGN KEY trong câu lệnh CREATE TABLE chỉ mang ý nghĩa tài liệu chứ không có tác dụng chặn dữ liệu sai [2].

2.2. Hệ quản trị cơ sở dữ liệu SQLite

SQLite là hệ quản trị cơ sở dữ liệu nhúng: toàn bộ cơ sở dữ liệu nằm trong một tệp duy nhất, không có tiến trình máy chủ, không cần cấu hình và không phát sinh chi phí bản quyền. Thư viện này có sẵn trong thư viện chuẩn của Python thông qua mô-đun sqlite3, nhờ đó một chương trình Python có thể vừa tạo, vừa truy vấn cơ sở dữ liệu mà không cần cài đặt thêm gói nào [4].

SQLite dùng hệ kiểu dữ liệu động (type affinity): kiểu khai báo chỉ là gợi ý, giá trị thực tế được lưu theo kiểu phù hợp. Vì không có kiểu ngày riêng, cột ngày thường được lưu dạng chuỗi YYYY-MM-DD — cách này vẫn cho phép so sánh, sắp xếp đúng thứ tự thời gian và dùng được hàm strftime() để nhóm theo tháng.

2.3. Ngôn ngữ SQL và các kỹ thuật truy vấn

2.3.1. Nhóm lệnh DDL, DML và DQL

SQL được chia thành ba nhóm lệnh chính dùng trong đồ án: DDL (CREATE TABLE, CREATE INDEX) định nghĩa cấu trúc; DML (INSERT, UPDATE, DELETE) thay đổi dữ liệu; DQL (SELECT) truy vấn dữ liệu. Mỗi nhóm có ngữ nghĩa giao dịch riêng: DML cần commit để ghi thay đổi vĩnh viễn, còn DDL trong SQLite cũng tham gia giao dịch.

2.3.2. Kết nối bảng (JOIN)

Phép nối kết hợp các hàng của hai bảng dựa trên điều kiện liên quan. INNER JOIN chỉ giữ các cặp hàng thoả điều kiện, trong khi LEFT JOIN giữ toàn bộ hàng của bảng bên trái và điền NULL cho phần không khớp. Chính khác biệt này khiến LEFT JOIN trở thành công cụ chuẩn để tìm các bản ghi "không có quan hệ", ví dụ khách hàng chưa từng đặt hàng.

2.3.3. Tổng hợp nhóm (GROUP BY và HAVING)

GROUP BY gom các hàng thành nhóm theo giá trị của một hoặc nhiều cột, sau đó các hàm tổng hợp như COUNT, SUM, AVG tính toán trên từng nhóm. WHERE lọc ở mức từng hàng trước khi gom nhóm, còn HAVING lọc ở mức nhóm sau khi đã tổng hợp — vì vậy muốn lọc theo doanh thu tính bằng SUM thì bắt buộc phải dùng HAVING.

2.3.4. Truy vấn con (subquery)

Truy vấn con là một câu SELECT lồng trong câu khác. Nếu truy vấn con trả về một giá trị duy nhất (truy vấn vô hướng), nó có thể dùng trực tiếp trong biểu thức so sánh, ví dụ tìm sản phẩm có giá cao hơn giá trung bình. Truy vấn con không tương quan như trường hợp này chỉ cần tính một lần, nên chi phí không tăng theo số hàng của truy vấn ngoài.

2.3.5. Hàm cửa sổ (window function)

Hàm cửa sổ tính toán trên một tập hàng liên quan nhưng không làm mất chi tiết từng hàng, khác với GROUP BY. RANK() trả về thứ hạng có bỏ qua các hạng trùng (1, 2, 2, 4), còn DENSE_RANK() không bỏ qua (1, 2, 2, 3). SQLite hỗ trợ hàm cửa sổ từ phiên bản 3.25 (2018) [7].

2.4. Chỉ mục và kế hoạch thực thi

2.4.1. Cấu trúc B-tree

Chỉ mục (index) là một cấu trúc dữ liệu phụ trợ giúp tìm hàng nhanh hơn. SQLite cài đặt chỉ mục dưới dạng cây B-tree: mỗi nút chứa nhiều khoá đã sắp xếp và con trỏ tới nút con, cho phép tìm kiếm theo thời gian tỉ lệ với logarit của số hàng thay vì quét toàn bảng. Đổi lại, mỗi chỉ mục chiếm thêm dung lượng và làm chậm các thao tác ghi vì phải cập nhật cây.

2.4.2. EXPLAIN QUERY PLAN

Lệnh EXPLAIN QUERY PLAN cho biết cách SQLite sẽ thực hiện một truy vấn mà không thực sự chạy nó [1]. Kết quả thường gặp gồm SCAN (đọc tuần tự toàn bảng), SEARCH (tìm qua chỉ mục hoặc khoá chính), USE TEMP B-TREE FOR ORDER BY (phải sắp xếp thêm bằng cấu trúc tạm) và COVERING INDEX (chỉ mục đã chứa đủ cột cần đọc nên không phải quay lại bảng).

2.4.3. Covering index

Khi mọi cột mà truy vấn cần đọc đều nằm trong chỉ mục, SQLite trả lời hoàn toàn từ chỉ mục. Trong SQLite, bảng có khoá chính INTEGER được lưu theo rowid, nên chỉ mục trên một cột bất kỳ thực chất chứa cả rowid — đây là lý do truy vấn chỉ lấy order_id theo customer_id có thể dùng covering index.

2.5. Môi trường thực hành: Google Cloud Shell

Google Cloud Shell là môi trường dòng lệnh chạy trên trình duyệt, cung cấp sẵn Python, các công cụ phát triển và 5 GB lưu trữ trong thư mục $HOME mà không cần thanh toán [5]. Môi trường này phù hợp với bài thực hành vì không phải cài đặt gì, đồng thời vẫn cho phép lưu lại mã nguồn và tệp cơ sở dữ liệu giữa các phiên làm việc. Hạn chế cần lưu ý là vùng lưu trữ có thể bị thu hồi nếu thời gian dài không đăng nhập, nên mã nguồn và cơ sở dữ liệu cần được sao lưu về máy cá nhân hoặc kho lưu trữ Git.

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ

3.1. Yêu cầu của bài toán

Đề bài đặt ra 21 yêu cầu, chia thành bốn nhóm theo mức độ và theo điều kiện dữ liệu. Bảng 3.1 liệt kê toàn bộ yêu cầu và kỹ thuật SQL tương ứng được sử dụng.

Bảng 3.1. Danh sách 21 yêu cầu của đề bài

Mã	Yêu cầu	Kỹ thuật chính
S01	Hiển thị tên và thành phố khách hàng	SELECT, ORDER BY
S02	Sản phẩm có giá trên 1 triệu	WHERE so sánh số
S03	Sắp xếp sản phẩm theo giá giảm dần	ORDER BY DESC
S04	Thêm một khách hàng mới	INSERT ... ON CONFLICT
S05	Tăng giá Mouse thêm 10%	UPDATE có điều kiện
S06	Xoá khách hàng vừa thêm	DELETE + NOT EXISTS
S07	Liệt kê mã đơn và tên khách hàng	INNER JOIN
S08	Tính tổng tiền từng đơn hàng	JOIN 3 bảng, GROUP BY, SUM
S09	Doanh thu từng khách hàng	LEFT JOIN, GROUP BY
S10	Khách chi tiêu trên 5 triệu	HAVING SUM(...) > ngưỡng
S11	Sản phẩm giá cao hơn trung bình	Truy vấn con vô hướng
S12	Khách có từ 2 đơn hàng	GROUP BY + HAVING COUNT
S13	Top 3 sản phẩm theo doanh thu	ORDER BY + LIMIT
S14	Tạo chỉ mục trên orders.customer_id	CREATE INDEX
S15	Xem kế hoạch thực thi trước và sau chỉ mục	EXPLAIN QUERY PLAN
S16	Doanh thu theo tháng	strftime + GROUP BY
S17	Khách chưa có đơn hàng	LEFT JOIN ... IS NULL
S18	Sản phẩm chưa từng bán	NOT EXISTS
S19	Xếp hạng khách theo doanh thu	RANK, DENSE_RANK
S20	So sánh kế hoạch thực thi sau khi mở rộng dữ liệu	EXPLAIN trên dữ liệu lớn
S21	So sánh bảng tạm SQL Server và SQLite	TEMP TABLE, kiến thức hệ quản trị

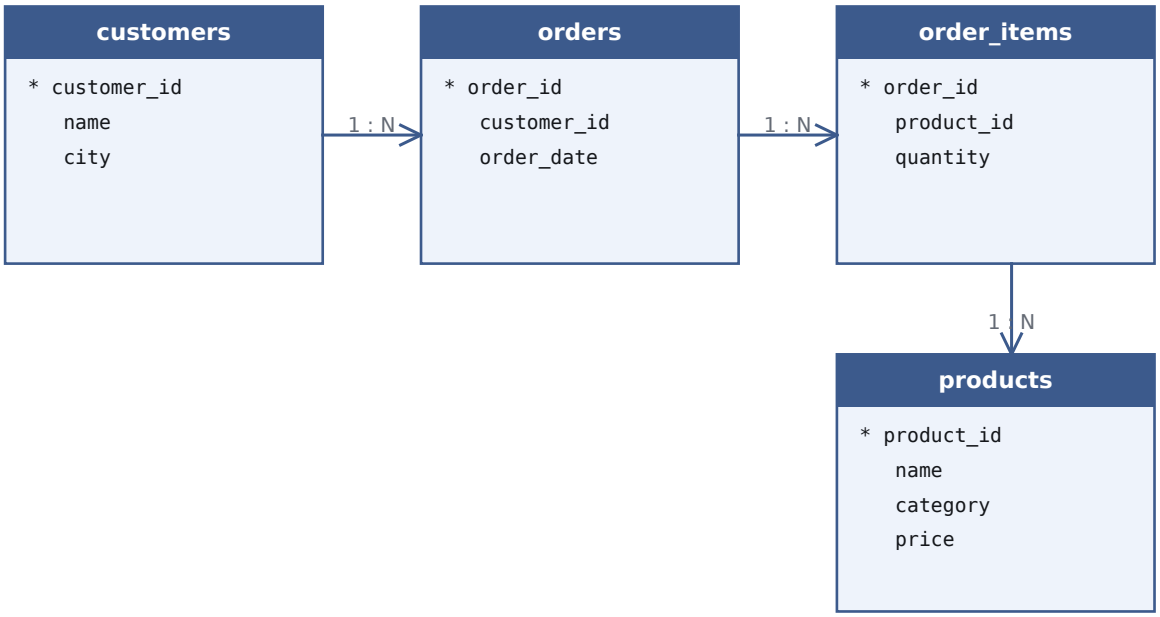
Một điểm cần phân tích kỹ là điều kiện dữ liệu: nhóm S01–S15 chạy trên bộ dữ liệu mẫu nhỏ (5 khách, 5 sản phẩm, 6 đơn), còn nhóm S16–S21 cần bộ dữ liệu lớn hơn nhiều. Nếu đặt tất cả trong một tệp đáp án duy nhất thì người chấm buộc phải tự chuyển đổi cơ sở dữ liệu giữa hai phần. Vì vậy đồ án tách thành hai tệp riêng và ghi rõ tệp nào chạy trên cơ sở dữ liệu nào, như trình bày ở mục 3.3.

3.2. Thiết kế mô hình dữ liệu

3.2.1. Sơ đồ quan hệ

Cơ sở dữ liệu gồm bốn bảng với ba quan hệ một–nhiều: một khách

hàng có nhiều đơn hàng, một đơn hàng có nhiều dòng đơn, một sản phẩm có thể xuất hiện trong nhiều dòng đơn. Sơ đồ được thể hiện ở Hình 3.1.



Hình 3.1. Sơ đồ quan hệ của cơ sở dữ liệu bán hàng

3.2.2. Chi tiết các bảng

Bảng 3.2 mô tả chi tiết từng cột, kiểu dữ liệu và ràng buộc của bốn bảng. Các ràng buộc được đặt ngay trong câu lệnh CREATE TABLE để cơ sở dữ liệu tự bảo vệ tính toàn vẹn, không phụ thuộc vào việc chương trình ứng dụng có kiểm tra hay không.

Bảng 3.2. Chi tiết các bảng trong cơ sở dữ liệu

Bảng	Cột	Kiểu	Ràng buộc
customers	customer_id	INTEGER	PRIMARY KEY
	name	TEXT	NOT NULL
	city	TEXT	—
products	product_id	INTEGER	PRIMARY KEY
	name	TEXT	NOT NULL
	category	TEXT	—
	price	REAL	CHECK(price >= 0)
orders	order_id	INTEGER	PRIMARY KEY
	customer_id	INTEGER	NOT NULL, FK → customers
	order_date	TEXT	—
order_items	order_id	INTEGER	PK tổ hợp, FK → orders
	product_id	INTEGER	PK tổ hợp, FK → products
	quantity	INTEGER	CHECK(quantity > 0)

3.2.3. Các quyết định thiết kế

Ba quyết định thiết kế đáng chú ý trong lược đồ:

- Khoá chính của order_items là tổ hợp (order_id, product_id). Nhờ vậy mỗi sản phẩm chỉ xuất hiện một lần trong một đơn; muốn mua thêm thì tăng giá trị quantity thay vì thêm dòng mới. Thiết kế này cũng tránh được dữ liệu trùng lặp.
- Ràng buộc CHECK(price >= 0) và CHECK(quantity > 0) chặn ngay tại tầng cơ sở dữ liệu các giá trị vô nghĩa như giá âm hoặc số lượng bằng không.
- Cột order_date lưu dạng TEXT theo chuỗi YYYY-MM-DD, phù hợp với SQLite (không có kiểu ngày riêng) và vẫn cho phép sắp xếp, so sánh, đồng thời dùng được strftime() để nhóm theo tháng cho yêu cầu S16.

3.3. Thiết kế kiến trúc chương trình

Thay vì dồn toàn bộ thao tác vào một tệp, đồ án tổ chức thành các tệp có trách nhiệm riêng, mỗi tệp chạy độc lập từ dòng lệnh. Cách tổ chức này giúp dễ kiểm thử, dễ chạy lại từng bước và phù hợp với yêu cầu nộp kèm mã nguồn của đề bài. Bảng 3.3 mô tả các tệp chính.

Bảng 3.3. Kiến trúc các tệp mã nguồn

Tệp	Trách nhiệm
sql/schema.sql	Định nghĩa bốn bảng, khoá và ràng buộc
sql/seed.sql	Nạp bộ dữ liệu mẫu đúng theo đề bài
sql/answers.sql	Đáp án nhóm bắt buộc S01–S15 (chạy trên sales.db)
sql/answers_homework.sql	Đáp án nhóm mở rộng S16–S21 (chạy trên sales_extended.db)
sql/explain.sql	Cặp EXPLAIN QUERY PLAN trước và sau khi tạo chỉ mục
build_db.py	Dựng sales.db từ schema và seed, in số dòng để đối chiếu
generate_data.py	Sinh dữ liệu mở rộng 100/30/500/1500 với seed cố định
run_sql.py	Chạy tệp .sql bằng Python, in kết quả từng câu lệnh
sql_console.py	Console SQLite tương tác theo mẫu của đề bài

Luồng thực hiện tổng quát gồm bốn bước: dựng cơ sở dữ liệu và nạp dữ liệu mẫu; chạy tệp đáp án phần bắt buộc; sinh dữ liệu mở rộng và chạy tệp đáp án phần bổ sung; chạy tệp kế hoạch thực thi để thu thập bằng chứng về chỉ mục.

3.4. Thiết kế dữ liệu kiểm thử

Đồ án sử dụng hai bộ dữ liệu. Bộ thứ nhất là dữ liệu mẫu của đề bài, đủ nhỏ để có thể kiểm tra kết quả bằng tay. Bộ thứ hai là dữ liệu

sinh tự động, đủ lớn để quan sát sự khác biệt về kế hoạch thực thi. Bảng 3.4 tổng hợp quy mô hai bộ dữ liệu.

Bảng 3.4. Quy mô hai bộ dữ liệu kiểm thử

Đối tượng	Dữ liệu mẫu (đề bài)	Dữ liệu mở rộng	Yêu cầu tối thiểu
Khách hàng	5	100	100
Sản phẩm	5	30	30
Đơn hàng	6	500	500
Dòng đơn hàng	8	1.500	1.000
Khoảng thời gian	01–08/10/2026	10/2025 – 10/2026	—

Dữ liệu mở rộng được sinh bằng bộ sinh số giả ngẫu nhiên có hạt giống cố định (random.seed(2026)), nên chạy lại cho đúng cùng một bộ dữ liệu — điều kiện cần để kết quả trong báo cáo có thể tái lập. Kịch bản sinh cố ý để khoảng 8% khách hàng không phát sinh đơn và bốn sản phẩm cuối danh mục không xuất hiện trong đơn nào, nhằm bảo đảm hai truy vấn S17 và S18 có kết quả để kiểm chứng.

CHƯƠNG 4. TRIỂN KHAI VÀ THỰC NGHIỆM

4.1. Môi trường thực hành

Bảng 4.1. Môi trường thực hành

Thành phần	Giá trị sử dụng
Nền tảng	Google Cloud Shell (chế độ lưu trữ $HOME), không bật dịch vụ tính phí
Ngôn ngữ	Python 3.11.2, chỉ dùng thư viện chuẩn sqlite3
Hệ quản trị	SQLite 3.40.1, cơ sở dữ liệu dạng tệp sales.db
Công cụ hỗ trợ	build_db.py, run_sql.py, generate_data.py, sql_console.py
Chi phí	0 đồng

4.2. Khởi tạo cơ sở dữ liệu

Lược đồ được viết trong schema.sql với bốn câu lệnh CREATE TABLE, mỗi câu khai báo đầy đủ khoá và ràng buộc như thiết kế ở Chương 3.

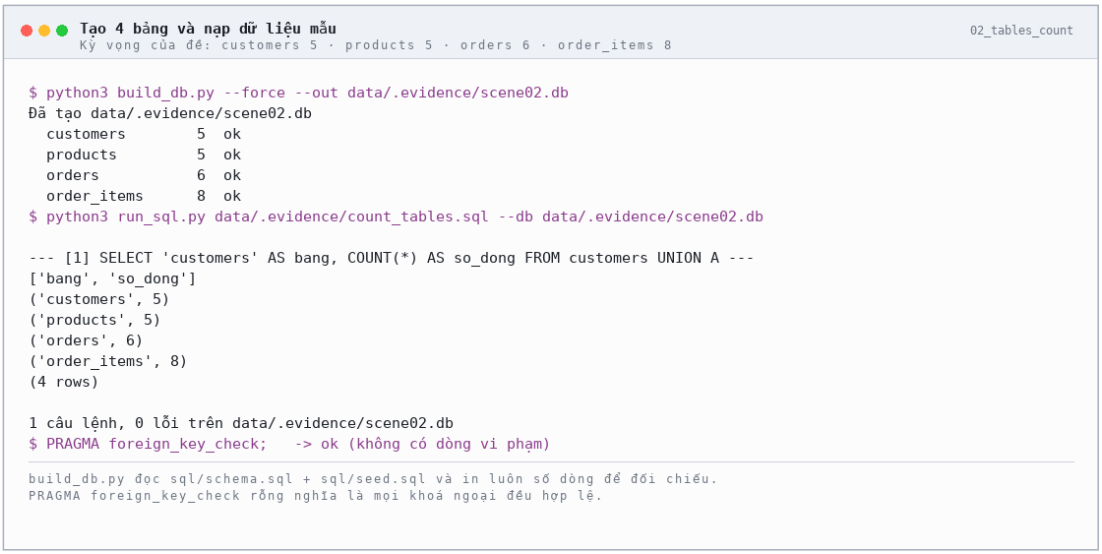
```
CREATE TABLE IF NOT EXISTS customers (
    customer_id INTEGER PRIMARY KEY,
    name        TEXT NOT NULL,
    city        TEXT
);

CREATE TABLE IF NOT EXISTS products (
    product_id INTEGER PRIMARY KEY,
    name       TEXT NOT NULL,
    category   TEXT,
    price      REAL CHECK(price >= 0)
);

CREATE TABLE IF NOT EXISTS orders (
    order_id    INTEGER PRIMARY KEY,
    customer_id INTEGER NOT NULL,
    order_date  TEXT,
    FOREIGN KEY(customer_id) REFERENCES customers(customer_id)
);

CREATE TABLE IF NOT EXISTS order_items (
    order_id   INTEGER,
    product_id INTEGER,
    quantity   INTEGER CHECK(quantity > 0),
    PRIMARY KEY(order_id, product_id),
    FOREIGN KEY(order id) REFERENCES orders(order id),
    FOREIGN KEY(product id) REFERENCES products(product id)
);
```

Dữ liệu mẫu được nạp bằng seed.sql. Để tệp có thể chạy lại an toàn, các câu lệnh dùng INSERT OR IGNORE: nếu bản ghi đã tồn tại thì bỏ qua thay vì báo lỗi trùng khoá chính. Sau khi nạp, kịch bản build_db.py đếm số dòng từng bảng và so sánh với kỳ vọng của đề bài (5, 5, 6, 8), đồng thời chạy PRAGMA foreign_key_check để phát hiện khoá ngoại mồ côi.



Hình 4.1. Kết quả khởi tạo: số dòng bốn bảng và kiểm tra khoá ngoại

4.3. Cài đặt nhóm truy vấn cơ bản và CRUD (S01–S07)

Nhóm đầu tiên gồm ba truy vấn đọc đơn giản và bốn thao tác có ghi. Ba thao tác ghi là điểm cần xử lý cẩn thận nhất, vì nếu viết ngây thơ thì mỗi lần chạy lại tệp đáp án sẽ làm sai dữ liệu: khách hàng bị thêm hai lần, giá sản phẩm bị nhân 10% nhiều lần. Đồ án áp dụng ba biện pháp tương ứng, trình bày ở đoạn mã dưới đây.

```
-- S04: them khach hang, chay lai khong loi khoa chinh
INSERT INTO customers (customer_id, name, city)
VALUES (6, 'Lan', 'Hue')
ON CONFLICT(customer_id) DO NOTHING;

-- S05: tang gia Mouse 10%, chi khi con o gia goc 250000
UPDATE products
SET price = ROUND(price * 1.1, 0)
WHERE product_id = 102 AND price = 250000;

-- S06: xoa dung khach vua them, va chi khi chua co don
DELETE FROM customers
WHERE customer_id = 6
  AND NOT EXISTS (SELECT 1 FROM orders
                  WHERE orders.customer_id = customers.customer_id);
```

Điểm mấu chốt là điều kiện AND price = 250000 trong câu UPDATE. Ở lần chạy thứ hai, giá Mouse đã là 275.000 nên câu lệnh không khớp dòng nào và trả về Rows affected: 0; nhờ vậy phép nhân 10% không bị cộng dồn. Tương tự, điều kiện NOT EXISTS trong câu DELETE vừa diễn đạt đúng yêu cầu “khách vừa thêm chưa có đơn hàng”, vừa bảo vệ khoá ngoại: nếu khách đã có đơn, câu lệnh không xoá gì thay vì gây lỗi ràng buộc.

Kết quả kiểm chứng sau hai lần chạy liên tiếp: giá Mouse giữ nguyên 275.000 và số khách hàng trở về 5. Truy vấn S07 dùng INNER JOIN để ghép orders với customers, trả về đủ 6 đơn hàng kèm tên khách.

4.4. Cài đặt truy vấn tổng hợp nhiều bảng (S08–S13)

S08 là truy vấn trung tâm của nhóm này: doanh thu của một đơn bằng tổng của tích số lượng và đơn giá trên từng dòng đơn, nên phải nối ba bảng orders, order_items và products rồi gom nhóm theo mã đơn.

```
-- S08: tong tien tung don hang
SELECT o.order_id,
       SUM(i.quantity * p.price) AS total
FROM orders o
JOIN order_items i ON i.order_id = o.order_id
JOIN products p    ON p.product_id = i.product_id
GROUP BY o.order_id
ORDER BY total DESC;

-- S10: khach chi tieu tren 5 trieu (loc theo ket qua tong hop)
SELECT c.customer_id, c.name,
       SUM(i.quantity * p.price) AS revenue
FROM customers c
JOIN orders o      ON o.customer_id = c.customer_id
JOIN order_items i ON i.order_id = o.order_id
JOIN products p    ON p.product_id = i.product_id
GROUP BY c.customer_id, c.name
HAVING SUM(i.quantity * p.price) > 5000000
ORDER BY revenue DESC;
```

Bảng 4.2. Kết quả S08 và S09 trên dữ liệu mẫu

Mã đơn	Tổng tiền (đồng)	Khách hàng	Doanh thu (đồng)
1001	15.550.000	An (id 1)	17.150.000
1004	15.000.000	Chi (id 3)	15.000.000
1006	7.000.000	Ha (id 5)	7.000.000
1002	4.150.000	Binh (id 2)	4.150.000
1003	1.600.000	Dung (id 4)	825.000
1005	825.000	—	—
Tổng	44.125.000	—	—

Có thể kiểm tra chéo bằng tay đối với đơn 1001: đơn gồm một Laptop giá 15.000.000 và hai Mouse giá 275.000, tổng 15.550.000 — khớp với kết quả máy trả về. Con số này cũng cho thấy thứ tự thực hiện các câu lệnh có ảnh hưởng tới kết quả: do S05 đã tăng giá Mouse trước khi chạy S08, tổng tiền dùng giá 275.000 chứ không phải 250.000. Báo cáo ghi rõ điểm này để tránh nhầm lẫn khi đối chiếu số liệu.



Hình 4.2. Kết quả truy vấn S08 và S09 trên dữ liệu mẫu

Ba truy vấn còn lại thể hiện ba kỹ thuật khác nhau. S11 dùng truy vấn con vô hướng để so sánh với giá trung bình; sau khi S05 đổi giá, giá trung bình của 5 sản phẩm là 4.045.000 nên chỉ Laptop (15.000.000) vượt trung bình. S12 đếm số đơn theo khách rồi lọc bằng HAVING COUNT(o.order_id) >= 2, kết quả đúng một khách là An với hai đơn. S13 sắp xếp giảm dần theo doanh thu và lấy LIMIT 3, cho ba sản phẩm dẫn đầu là Laptop (30.000.000), Monitor (10.500.000) và Headphones (1.600.000).

> **Hạn chế của LIMIT khi có đồng hạng**
> S13 dùng LIMIT 3 nên nếu hai sản phẩm cùng đứng ở vị trí thứ ba, kết quả chỉ trả về một trong hai. Muốn xử lý đúng trường hợp đồng hạng phải dùng hàm cửa sổ RANK() như ở yêu cầu S19. Đây là ví dụ cho thấy cùng một câu hỏi nghiệp vụ có thể cần kỹ thuật phức tạp hơn tuỳ yêu cầu về tính đầy đủ của kết quả.

4.5. Chỉ mục và phân tích kế hoạch thực thi (S14–S15)

Yêu cầu S15 đòi hỏi so sánh kế hoạch thực thi trước và sau khi có chỉ mục, nên không thể chỉ chạy một lần sau khi đã tạo chỉ mục. Tệp explain.sql được viết để tự đưa cơ sở dữ liệu về trạng thái cần thiết theo trình tự: xoá chỉ mục nếu tồn tại, xem kế hoạch, tạo chỉ mục, rồi xem kế hoạch lần hai. Nhờ vậy tệp chạy đúng trên bất kỳ bản sao nào của cơ sở dữ liệu, ở bất kỳ trạng thái nào.

```
DROP INDEX IF EXISTS idx_orders_customer;
EXPLAIN QUERY PLAN
SELECT order_id FROM orders WHERE customer_id = 1;

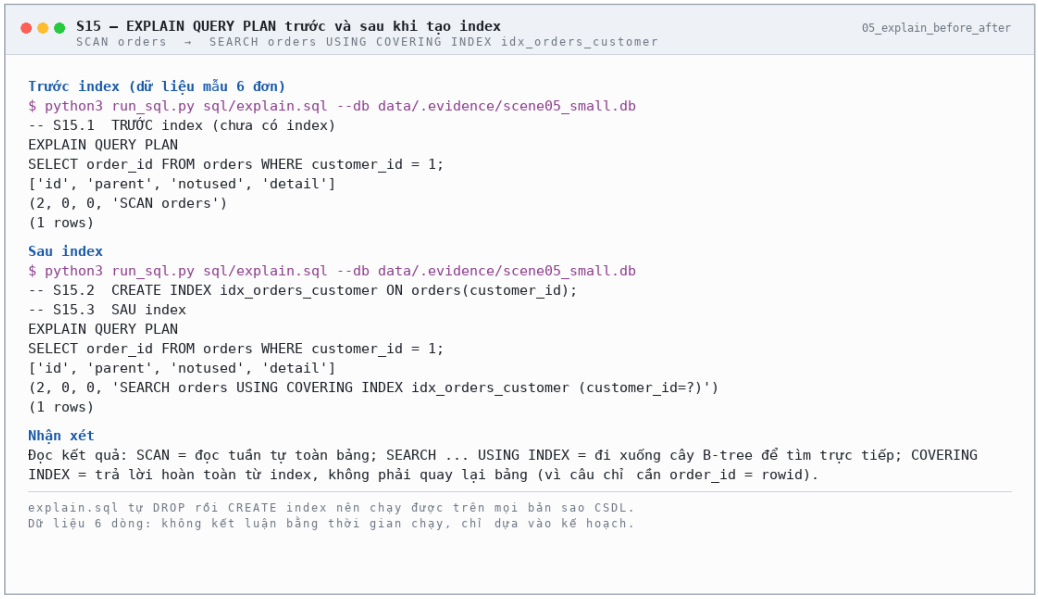
CREATE INDEX IF NOT EXISTS idx_orders_customer
    ON orders(customer_id);
```

```
EXPLAIN QUERY PLAN
SELECT order_id FROM orders WHERE customer_id = 1;
```

Bảng 4.3. Kế hoạch thực thi trước và sau khi tạo chỉ mục

Thời điểm	Kế hoạch thực thi (cột detail)
Trước khi tạo chỉ mục	SCAN orders
Sau khi tạo chỉ mục	SEARCH orders USING COVERING INDEX idx_orders_customer (customer_id=?)

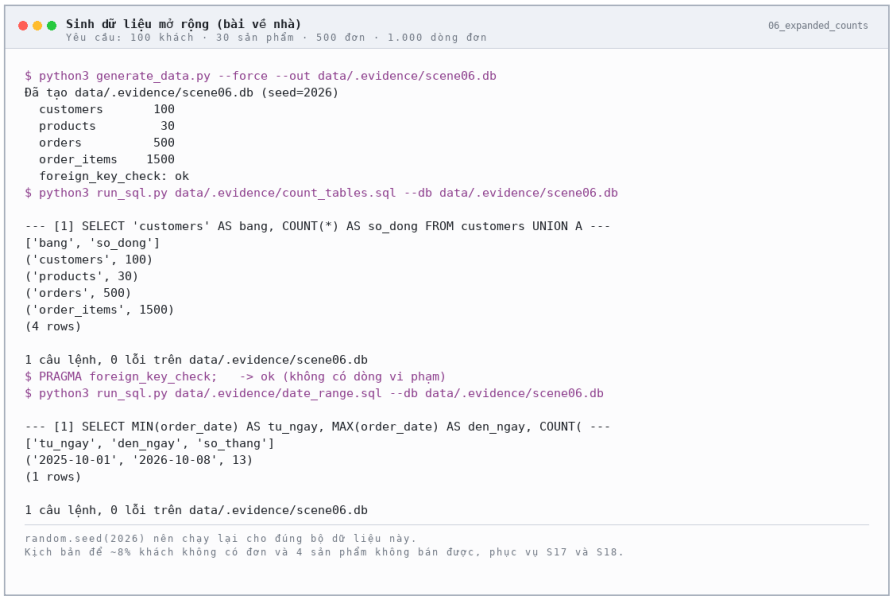
Kết quả cho thấy hai thay đổi. Thứ nhất, kiểu truy cập đổi từ SCAN — đọc tuần tự toàn bảng rồi mới lọc — sang SEARCH, tức đi xuống cây B-tree của chỉ mục để tìm trực tiếp các dòng cần thiết. Thứ hai, kế hoạch ghi COVERING INDEX: câu truy vấn chỉ cần cột order_id, mà chỉ mục trên customer_id của bảng có khoá chính INTEGER đã chứa sẵn rowid (chính là order_id), nên SQLite trả lời hoàn toàn từ chỉ mục mà không phải đọc lại bảng.



Hình 4.3. Kế hoạch thực thi trước và sau khi tạo chỉ mục trên cột customer_id

4.6. Sinh dữ liệu mở rộng và các yêu cầu S16–S21

Kịch bản generate_data.py sinh dữ liệu mở rộng vào một tệp riêng sales_extended.db, không ghi đè sales.db, để bộ dữ liệu mẫu vẫn giữ nguyên trạng thái ban đầu. Sau khi sinh, kịch bản in số dòng từng bảng và kiểm tra khoá ngoại; kết quả đạt 100 khách hàng, 30 sản phẩm, 500 đơn hàng và 1.500 dòng đơn hàng, vượt mức tối thiểu của đề bài.



Hình 4.4. Kết quả sinh dữ liệu mở rộng và kiểm tra tính toàn vẹn

Trên bộ dữ liệu mở rộng, yêu cầu S16 tính doanh thu theo tháng bằng cách nhóm theo strftime('%Y-%m', order_date). Dữ liệu trải 13 tháng nên kết quả có 13 nhóm, trong đó nhóm cuối chỉ có 7 đơn vì dữ liệu kết thúc giữa tháng — hiện tượng "tháng dở dang" cần nêu rõ khi báo cáo doanh thu. Bảng 4.4 trình bày kết quả đầy đủ.

Bảng 4.4. Doanh thu theo tháng trên dữ liệu mở rộng (S16)

Tháng	Số đơn	Doanh thu (đồng)
2025-10	53	1.142.485.000
2025-11	39	978.725.000
2025-12	36	797.655.000
2026-01	38	780.995.000
2026-02	37	870.780.000
2026-03	46	1.051.310.000
2026-04	47	947.890.000
2026-05	42	838.505.000
2026-06	35	813.970.000
2026-07	40	696.115.000
2026-08	44	899.005.000
2026-09	36	853.310.000
2026-10	7	48.400.000

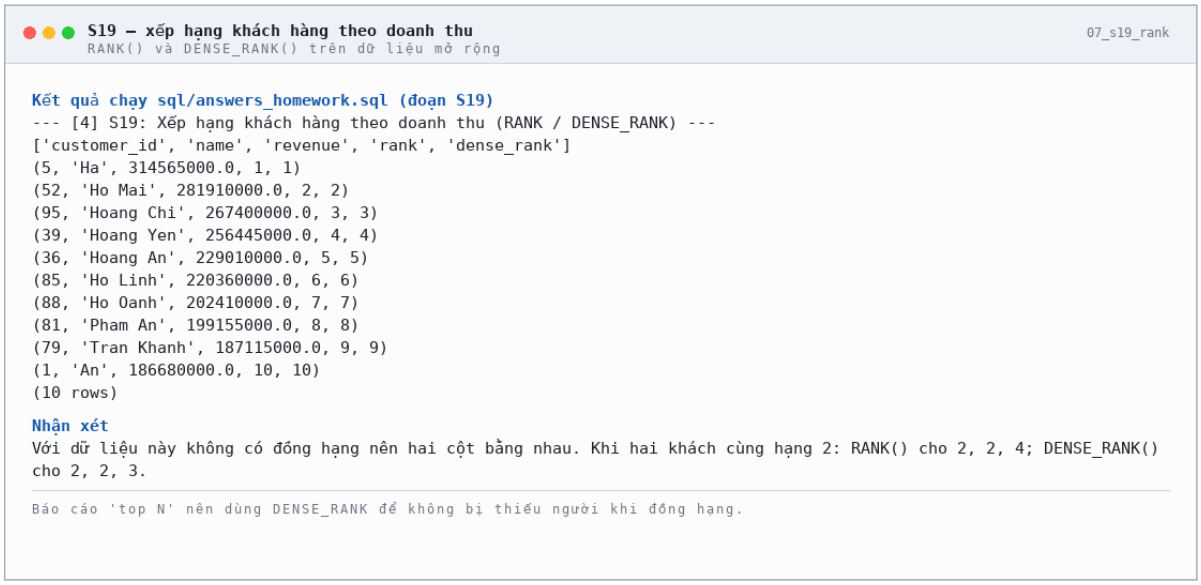
Hai yêu cầu S17 và S18 đều là bài toán tìm bản ghi không có quan hệ tương ứng, nhưng được cài đặt bằng hai kỹ thuật khác nhau để so sánh: S17 dùng LEFT JOIN kết hợp điều kiện IS NULL, S18 dùng NOT EXISTS. Cả hai cho kế hoạch thực thi tương đương trên SQLite; NOT EXISTS thường được ưu tiên vì diễn đạt trực tiếp ngữ nghĩa và không sinh thêm cột NULL trung gian. Kết quả tìm được 8 khách hàng chưa có đơn (id 37, 38, 43, 56, 83, 89, 92, 94) và 4 sản

phẩm chưa từng bán (Graphics Card, RAM 16GB, CPU Cooler, Power Supply).

```
-- S17: khach hang chua co don hang nao
SELECT c.customer_id, c.name, c.city
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL
ORDER BY c.customer_id;

-- S19: xep hang khach hang theo doanh thu
SELECT c.customer_id, c.name,
       ROUND(SUM(i.quantity * p.price), 0) AS revenue,
       RANK()       OVER (ORDER BY SUM(i.quantity * p.price) DESC) AS rank,
       DENSE_RANK() OVER (ORDER BY SUM(i.quantity * p.price) DESC) AS dense_rank
FROM customers c
JOIN orders o      ON o.customer id = c.customer id
JOIN order items i ON i.order id = o.order id
JOIN products p    ON p.product id = i.product id
GROUP BY c.customer_id, c.name
ORDER BY revenue DESC;
```

Yêu cầu S19 xếp hạng khách hàng bằng hàm cửa sổ. Trên bộ dữ liệu mở rộng, ba vị trí dẫn đầu là Ha (314.565.000 đồng), Ho Mai (281.910.000 đồng) và Hoang Chi (267.400.000 đồng). Do không có hai khách nào trùng doanh thu, hai cột rank và dense_rank trùng nhau; khác biệt chỉ xuất hiện khi có đồng hạng — khi đó RANK nhảy từ 2 sang 4 còn DENSE_RANK đi liền sang 3.



Hình 4.5. Kết quả xếp hạng khách hàng theo doanh thu với RANK và DENSE_RANK

Yêu cầu S20 lặp lại phân tích kế hoạch thực thi trên bộ dữ liệu lớn. Kết quả vẫn là SCAN khi chưa có chỉ mục và SEARCH khi có chỉ mục, nhưng lần này xuất hiện thêm dòng USE TEMP B-TREE FOR ORDER BY vì truy vấn lấy thêm cột order_date và sắp xếp theo cột này — cột không nằm trong chỉ mục nên SQLite phải dựng cấu trúc sắp xếp tạm. Nếu truy vấn được chạy thường xuyên, một chỉ mục tổ hợp trên (customer_id, order_date) sẽ loại bỏ được bước sắp xếp

này.

Yêu cầu S21 là câu hỏi lý thuyết, yêu cầu so sánh bảng tạm của SQL Server với TEMP TABLE của SQLite. Bảng 4.5 tổng hợp nội dung so sánh.

Bảng 4.5. So sánh bảng tạm giữa SQL Server và SQLite (S21)

Tiêu chí	SQL Server	SQLite
Bảng tạm cục bộ	#t — chỉ session tạo ra nó thấy, tự xoá khi session kết thúc	CREATE TEMP TABLE — gắn với kết nối, mất khi đóng
Bảng tạm toàn cục	##t — mọi session đều thấy, mất khi session cuối cùng đóng	Không có khái niệm tương đương
Nơi lưu trữ	tempdb — cơ sở dữ liệu hệ thống dùng chung, ghi nhật ký đầy đủ	Schema temp riêng, thường ở RAM hoặc tệp tạm tuỳ PRA
Phạm vi tên	Phân biệt bằng tiền tố # nên không đụng tên với bảng thường	Bảng temp được tìm trước main nên trùng tên sẽ che bả
Cú pháp xoá	DROP TABLE #t	DROP TABLE temp.t

4.7. Kiểm thử

Sau khi hoàn thiện, đồ án thực hiện các kiểm thử tổng hợp được liệt kê trong Bảng 4.6. Mỗi kiểm thử đều có tiêu chí đạt rõ ràng để có thể chạy lại và xác nhận.

Bảng 4.6. Các kiểm thử đã thực hiện

Kiểm thử	Cách thực hiện	Kết quả
Tạo dữ liệu mẫu	python3 build_db.py --force	5 / 5 / 6 / 8 dòng, khớp đề bài
Chạy lặp đáp án	Chạy answers.sql hai lần liên tiếp	0 lỗi, giá Mouse vẫn 275.000
Toàn vẹn khoá ngoại	PRAGMA foreign_key_check sau khi nạp	Không có dòng vi phạm
Dữ liệu mở rộng	python3 generate_data.py --force	100 / 30 / 500 / 1500 dòng
Đáp án mở rộng	Chạy answers_homework.sql	9 câu lệnh, 0 lỗi
Kế hoạch thực thi	Chạy explain.sql	SCAN chuyển thành SEARCH dùng chỉ mục
Console tương tác	Nhập câu lệnh nhiều dòng, câu sai, EXIT	Báo lỗi đúng, không thoát

CHƯƠNG 5. KẾT QUẢ VÀ ĐÁNH GIÁ

5.1. Kết quả đạt được theo tiêu chí

Bảng 5.1 đối chiếu kết quả thực hiện với năm tiêu chí chấm điểm của đề bài, kèm vị trí trình bày trong báo cáo để thuận tiện đối chiếu khi đánh giá.

Bảng 5.1. Đối chiếu kết quả với tiêu chí chấm điểm của đề bài

Tiêu chí	Điểm	Nội dung đã thực hiện	Vị trí
Cài đặt và tạo dữ liệu	2,0	Lược đồ 4 bảng có ràng buộc; dữ liệu mẫu đúng 5/5/8/12	Mục 4.2
CRUD và câu cơ bản	2,0	S01–S07 đầy đủ, CRUD chạy lặp an toàn	Mục 4.3
Truy vấn nâng cao	3,0	S08–S13 với JOIN nhiều bảng, HAVING, truy vấn con lồng; S16–S19 với window fu	Mục 4.4, 4.5
Index và giải thích	1,5	S14–S15, phân tích SCAN/SEARCH/COVERING INDEX, giải thích vì sao không đo bằ	Mục 4.5, 5.2
Bài mở rộng, báo cáo	1,5	Dữ liệu 100/30/500/1500 tái lập được; S16–S21 đầy đủ; báo cáo có phân tích và minh chứng	Chương 4, Phụ lục C

5.2. Đánh giá hiệu quả của chỉ mục

Kết quả thực nghiệm cho thấy chỉ mục thay đổi rõ rệt kế hoạch thực thi, nhưng không đo được khác biệt về thời gian trên bộ dữ liệu nhỏ. Bảng 5.2 tổng hợp hai kiểu truy cập và ý nghĩa của chúng.

Bảng 5.2. So sánh các kiểu truy cập dữ liệu trong SQLite

Kiểu truy cập	Cách thực hiện	Chi phí	Khi nào xuất hiện
SCAN	Đọc tuần tự toàn bảng rồi lọc	Tỉ lệ với số hàng của bảng	Không có chỉ mục phù hợp với điều kiện
SEARCH (index)	Đi xuống cây B-tree để tìm trực tiếp	Tỉ lệ với log(số hàng) cộng số hàng khớp	Có chỉ mục trên cột trong điều kiện lọc
SEARCH (covering)	Đọc hoàn toàn từ chỉ mục	Như SEARCH, không phải đọc bảng	Mọi cột cần đọc đều có trong chỉ mục

Với bảng 6 dòng, chênh lệch thời gian giữa SCAN và SEARCH nằm dưới ngưỡng đo của đồng hồ hệ thống, nên nếu đánh giá bằng thời gian chạy sẽ dễ kết luận sai rằng chỉ mục không có tác dụng. Cách đánh giá đúng là đọc kế hoạch thực thi: kiểu truy cập, số bước thực hiện và sự xuất hiện của các bước phụ như sắp xếp tạm. Đây cũng là lưu ý mà đề bài nêu rõ.

Một quan sát khác là chỉ mục không miễn phí. Mỗi chỉ mục làm tăng dung lượng tệp cơ sở dữ liệu và làm chậm các thao tác ghi, vì cây B-tree phải được cập nhật. Vì vậy chỉ nên tạo chỉ mục cho các cột thường xuyên xuất hiện trong điều kiện lọc hoặc điều kiện nối, như orders.customer_id trong bài toán này, thay vì tạo cho mọi cột.

5.3. Đánh giá tính đúng đắn và khả năng tái lập

Ba yếu tố sau bảo đảm kết quả trong báo cáo có thể kiểm chứng lại. Thứ nhất, dữ liệu mẫu được nạp từ tệp seed.sql có nội dung đúng

như đề bài, và kịch bản dựng cơ sở dữ liệu in ra số dòng để so sánh trực tiếp với kỳ vọng. Thứ hai, dữ liệu mở rộng được sinh với hạt giống cố định nên chạy lại cho đúng cùng một bộ dữ liệu. Thứ ba, các tệp đáp án được viết để chạy lặp an toàn, nên người đánh giá có thể chạy nhiều lần mà không làm hỏng trạng thái dữ liệu.

Ngoài ra, phần kết quả đều được đối chiếu chéo: tổng tiền từng đơn trong S08 được kiểm tra bằng tay với đơn 1001; tổng doanh thu theo khách trong S09 được so với tổng theo đơn; số khách chưa có đơn ở S17 khớp với số khách được kịch bản sinh cố ý bỏ trống.

5.4. Hạn chế

- Bộ dữ liệu mẫu quá nhỏ để đánh giá định lượng hiệu năng, nên các kết luận về chỉ mục chủ yếu mang tính định tính dựa trên kế hoạch thực thi.
- Dữ liệu mở rộng vẫn ở mức vài nghìn dòng, chưa đủ lớn để quan sát rõ ảnh hưởng của chỉ mục tổ hợp hoặc ảnh hưởng của việc sắp xếp tạm.
- Đồ án chưa có giao diện người dùng hay báo cáo trực quan hoá; kết quả được trình bày dưới dạng bảng và ảnh chụp màn hình.
- Chưa xử lý các tình huống đồng thời (nhiều tiến trình cùng ghi) vì SQLite ở cấu hình mặc định chỉ cho phép một tiến trình ghi tại một thời điểm.

5.5. Bài học kinh nghiệm

1. Khoá ngoại của SQLite là tuỳ chọn: phải bật PRAGMA foreign_keys = ON ở mọi kết nối, nếu không ràng buộc trong lược đồ chỉ mang tính tài liệu.
2. Kịch bản có thao tác ghi phải được thiết kế để chạy lặp an toàn bằng ON CONFLICT, điều kiện chặt trong UPDATE và NOT EXISTS trong DELETE.
3. Chỉ mục là con dao hai lưỡi: tăng tốc truy vấn đọc nhưng làm chậm ghi và tốn dung lượng, nên phải chọn cột để đánh chỉ mục dựa trên truy vấn thực tế.
4. Đọc kế hoạch thực thi là cách đánh giá ổn định hơn đo thời gian, nhất là khi dữ liệu nhỏ hoặc máy chạy khác nhau.
5. Báo cáo số liệu phải ghi rõ trạng thái dữ liệu tại thời điểm đo, vì thứ tự thực hiện các câu lệnh có ghi ảnh hưởng trực tiếp tới kết quả truy vấn sau đó.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết luận

Đồ án đã hoàn thành đầy đủ 21 yêu cầu của đề bài LAB 01. Cơ sở dữ liệu bán hàng gồm bốn bảng được thiết kế đúng theo mô hình quan hệ với khoá chính, khoá ngoại và ràng buộc miền giá trị; dữ liệu mẫu được nạp đúng 5 khách hàng, 5 sản phẩm, 6 đơn hàng và 8 dòng đơn hàng như kỳ vọng.

Về kỹ năng truy vấn, đồ án đã cài đặt và kiểm chứng các nhóm lệnh từ cơ bản đến nâng cao: truy vấn một bảng, thao tác dữ liệu, nối nhiều bảng, tổng hợp nhóm có lọc bằng HAVING, truy vấn con và hàm cửa sổ. Các thao tác có ghi đều được xử lý để chạy lặp an toàn, và điều này đã được kiểm chứng bằng cách chạy hai lần liên tiếp.

Về tối ưu truy vấn, đồ án đã chứng minh được bằng kế hoạch thực thi rằng việc tạo chỉ mục trên orders.customer_id chuyển truy vấn từ SCAN sang SEARCH, ở mức covering index; đồng thời chỉ ra giới hạn của việc suy luận hiệu năng trên dữ liệu nhỏ và cách khắc phục bằng cách đọc kế hoạch thay vì đo thời gian.

Về kỹ năng công cụ, đồ án đã sử dụng thành thạo Google Cloud Shell, Python và SQLite trong một quy trình làm việc hoàn toàn miễn phí, có thể tái lập và có kiểm thử tự động cho từng bước.

6.2. Hướng phát triển

- Bổ sung tầng giao diện: một ứng dụng web nhỏ hoặc giao diện dòng lệnh có menu để thao tác dữ liệu thay vì gõ SQL trực tiếp.
- Xây dựng các view và báo cáo tổng hợp (doanh thu theo tháng, theo thành phố, theo danh mục) để tái sử dụng thay vì viết lại truy vấn mỗi lần.
- Thử nghiệm chỉ mục tổ hợp trên (customer_id, order_date) và đo lại kế hoạch thực thi cho truy vấn ở yêu cầu S20 để loại bỏ bước sắp xếp tạm.
- Mở rộng quy mô dữ liệu lên hàng trăm nghìn dòng và đo thời gian thực thi thực tế để có kết luận định lượng về hiệu quả của chỉ mục.
- So sánh với mô hình tài liệu trên MongoDB ở LAB 02 để làm rõ đánh đổi giữa mô hình chuẩn hoá (quan hệ) và mô hình nhúng (tài liệu) đối với cùng một bài toán bán hàng.

TÀI LIỆU THAM KHẢO

[1] SQLite Consortium. EXPLAIN QUERY PLAN. https://www.sqlite.org/eqp.html

[2] SQLite Consortium. Foreign Key Support. https://www.sqlite.org/foreignkeys.html

[3] SQLite Consortium. CREATE TABLE — temporary tables. https://www.sqlite.org/lang_createtable.html

[4] Python Software Foundation. sqlite3 — DB-API 2.0 interface for SQLite databases. https://docs.python.org/3/library/sqlite3.html

[5] Google Cloud. Cloud Shell documentation. https://cloud.google.com/shell/docs

[6] Microsoft. CREATE TABLE (Transact-SQL) — temporary tables and tempdb. https://learn.microsoft.com/sql/t-sql/statements/create-table-transact-sql

[7] SQLite Consortium. Window Functions. https://www.sqlite.org/windowfunctions.html

[8] SQLite Consortium. Query Planning. https://www.sqlite.org/optoverview.html

CHƯƠNG A. CẤU TRÚC BÀI NỘP VÀ HƯỚNG DẪN TÁI LẬP

Bài nộp được đóng gói thành một tệp ZIP theo quy định của đề bài. Bảng A.1 mô tả nội dung từng thư mục và tệp trong gói nộp.

Bảng A.1. Cấu trúc gói bài nộp

Tệp / thư mục	Nội dung
data/sales.db	Cơ sở dữ liệu đã nạp dữ liệu mẫu
data/sales_extended.db	Cơ sở dữ liệu mở rộng dùng cho S16–S21
sql_console.py	Console SQLite tương tác (bắt buộc theo đề)
sql/answers.sql	Đáp án nhóm bắt buộc S01–S15
sql/answers_homework.sql	Đáp án nhóm mở rộng S16–S21
sql/explain.sql	Kế hoạch thực thi trước và sau chỉ mục
build_db.py, generate_data.py, run_sql.py	Các kịch bản hỗ trợ
report/*.pdf	Báo cáo đồ án (tệp này) và bản tóm tắt 3 trang
evidence/*.png	Ảnh minh chứng sinh từ kết quả chạy thật
README_MAPPING.txt	Bảng ánh xạ yêu cầu của đề sang tệp tương ứng

Để tái lập toàn bộ kết quả trong báo cáo, chạy lần lượt các lệnh dưới đây từ thư mục lab01_sql.

```
# 1) Du lieu mau theo de: ky vong 5 / 5 / 6 / 8
python3 build_db.py --force

# 2) Phan bat buoc S01-S15
python3 run_sql.py sql/answers.sql

# 3) Du lieu mo rong va phan bo sung S16-S21
python3 generate_data.py --force
python3 run_sql.py sql/answers_homework.sql --db data/sales_extended.db

# 4) Chung minh chi muc: SCAN orders -> SEARCH ... USING INDEX
python3 run_sql.py sql/explain.sql
python3 run_sql.py sql/explain.sql --db data/sales_extended.db

# 5) Sinh lai anh minh chung va bao cao
cd evidence && python3 make_evidence.py && cd ..
python3 report/make_report.py        # bao cao do an (tep nay)
python3 report/make_pdf.py           # ban tom tat 3 trang
```

CHƯƠNG B. MÃ NGUỒN CHÍNH

Phụ lục này trích nội dung hai tệp quan trọng nhất của bài làm: lược đồ cơ sở dữ liệu và toàn bộ đáp án nhóm bắt buộc. Các tệp còn lại nằm trong gói nộp kèm theo báo cáo.

B.1. Lược đồ cơ sở dữ liệu (schema.sql)

```
-- LAB 01 — Schema quan he cho cua hang truc tuyen
-- customers (1–N) orders, orders (1–N) order_items, products (1–N) order_items
-- Dung boi build_db.py. Chay lai nhieu lan van an toan vi co IF NOT EXISTS.

PRAGMA foreign keys = ON;

CREATE TABLE IF NOT EXISTS customers (
    customer_id INTEGER PRIMARY KEY,
    name        TEXT NOT NULL,
    city        TEXT
);

CREATE TABLE IF NOT EXISTS products (
    product_id INTEGER PRIMARY KEY,
    name       TEXT NOT NULL,
    category   TEXT,
    price      REAL CHECK(price >= 0)
);

CREATE TABLE IF NOT EXISTS orders (
    order_id    INTEGER PRIMARY KEY,
    customer_id INTEGER NOT NULL,
    order_date  TEXT,
    FOREIGN KEY(customer id) REFERENCES customers(customer id)
);

CREATE TABLE IF NOT EXISTS order_items (
    order id   INTEGER,
    product_id INTEGER,
    quantity   INTEGER CHECK(quantity > 0),
    PRIMARY KEY(order_id, product_id),
    FOREIGN KEY(order id) REFERENCES orders(order id),
    FOREIGN KEY(product_id) REFERENCES products(product_id)
);
```

B.2. Đáp án nhóm bắt buộc (answers.sql, S01–S15)

```
-- ======================================================================
-- LAB 01 — BAI TAP BAT BUOC (S01–S15)
-- Chay tren data/sales.db. File duoc viet de chay lai nhieu lan van sach:
--   S04 them khach id 6, S06 xoa dung khach do; S05 chi tang gia khi Mouse
--   con o gia goc 250.000 (khong cong don 10% moi lan chay).
--   python3 run sql.py sql/answers.sql
--   hoac: sqlite3 data/sales.db < sql/answers.sql
-- ======================================================================

-- S01: Hien thi ten va thanh pho khach hang
SELECT name, city
FROM customers
ORDER BY customer_id;

-- S02: San pham co gia > 1 trieu
SELECT product_id, name, price
FROM products
WHERE price > 1000000
ORDER BY price DESC;

-- S03: Sap xep san pham theo gia giam dan
SELECT product id, name, category, price
FROM products
```

```
ORDER BY price DESC;

-- S04: Them mot khach hang moi (Lan – Hue)
-- ON CONFLICT DO NOTHING de file chay lai khong bao loi khoa chinh.
INSERT INTO customers (customer id, name, city)
VALUES (6, 'Lan', 'Hue')
ON CONFLICT(customer id) DO NOTHING;

SELECT * FROM customers WHERE customer id = 6;

-- S05: Tang gia Mouse them 10% (250.000 -> 275.000)
-- Dieu kien price = 250000 giu phep nhan khong bi cong don khi chay lai file.
UPDATE products
SET price = ROUND(price * 1.1, 0)
WHERE product_id = 102 AND price = 250000;

SELECT product_id, name, price FROM products WHERE product_id = 102;

-- S06: Xoa khach hang vua them (chua co don hang nao)
DELETE FROM customers
WHERE customer id = 6
  AND NOT EXISTS (SELECT 1 FROM orders WHERE orders.customer id = customers.customer id);

SELECT COUNT(*) AS customers after delete FROM customers;

-- S07: Liet ke ma don va ten khach hang bang JOIN
SELECT o.order id, c.name AS customer name, o.order date
FROM orders o
JOIN customers c ON c.customer id = o.customer id
ORDER BY o.order id;

-- S08: Tinh tong tien tung don hang
SELECT o.order id,
       SUM(i.quantity * p.price) AS total
FROM orders o
JOIN order_items i ON i.order_id = o.order_id
JOIN products p    ON p.product_id = i.product_id
GROUP BY o.order_id
ORDER BY total DESC;

-- S09: Doanh thu tung khach hang
SELECT c.customer id,
       c.name,
       SUM(i.quantity * p.price) AS revenue
FROM customers c
LEFT JOIN orders o      ON o.customer id = c.customer id
LEFT JOIN order items i ON i.order id = o.order id
LEFT JOIN products p    ON p.product id = i.product id
GROUP BY c.customer id, c.name
ORDER BY revenue DESC;

-- S10: Khach hang chi tieu > 5 trieu (HAVING)
SELECT c.customer id,
       c.name,
       SUM(i.quantity * p.price) AS revenue
FROM customers c
JOIN orders o           ON o.customer_id = c.customer_id
JOIN order_items i      ON i.order_id = o.order_id
JOIN products p         ON p.product_id = i.product_id
GROUP BY c.customer_id, c.name
HAVING SUM(i.quantity * p.price) > 5000000
ORDER BY revenue DESC;

-- S11: San pham co gia cao hon gia trung binh (subquery)
SELECT product id, name, price
FROM products
WHERE price > (SELECT AVG(price) FROM products)
ORDER BY price DESC;

-- S12: Khach hang co tu 2 don hang
SELECT c.customer id, c.name, COUNT(o.order id) AS order count
FROM customers c
JOIN orders o ON o.customer id = c.customer id
GROUP BY c.customer id, c.name
HAVING COUNT(o.order id) >= 2
ORDER BY order_count DESC, c.customer_id;
```

```
-- S13: Top 3 san pham theo doanh thu
SELECT p.product id,
       p.name,
       SUM(i.quantity * p.price) AS revenue
FROM products p
JOIN order items i ON i.product id = p.product id
GROUP BY p.product id, p.name
ORDER BY revenue DESC
LIMIT 3;

-- S14: Tao index tren orders.customer id
CREATE INDEX IF NOT EXISTS idx orders customer ON orders(customer id);

-- S15: Xem EXPLAIN QUERY PLAN truoc va sau index
-- Phan "truoc index" can chay khi index chua ton tai, nen xem toan bo cap
-- before/after trong sql/explain.sql (file do tu DROP roi CREATE index).
-- O day chay sau index de thay SEARCH ... USING INDEX:
EXPLAIN QUERY PLAN
SELECT order_id FROM orders WHERE customer_id = 1;
```

B.3. Đáp án nhóm mở rộng (answers_homework.sql, S16–S21)

```
-- ======================================================================
-- LAB 01 — BAI TAP VE NHA / BO SUNG (S16—S21)
-- Chay tren ban du lieu mo rong do generate_data.py sinh ra:
--   python3 generate_data.py --force
--   python3 run_sql.py sql/answers_homework.sql --db data/sales_extended.db
-- (chay duoc ca tren data/sales.db, chi la so lieu nho hon)
-- ======================================================================

-- S16: Doanh thu theo thang
SELECT strftime('%Y-%m', o.order date) AS month,
       COUNT(DISTINCT o.order id)      AS orders,
       ROUND(SUM(i.quantity * p.price), 0) AS revenue
FROM orders o
JOIN order items i ON i.order id = o.order id
JOIN products p    ON p.product id = i.product id
GROUP BY month
ORDER BY month;

-- S17: Khach hang chua co don hang nao
SELECT c.customer id, c.name, c.city
FROM customers c
LEFT JOIN orders o ON o.customer id = c.customer id
WHERE o.order id IS NULL
ORDER BY c.customer id;

-- S18: San pham chua tung ban duoc
SELECT p.product_id, p.name, p.category, p.price
FROM products p
WHERE NOT EXISTS (
    SELECT 1 FROM order_items i WHERE i.product_id = p.product_id
)
ORDER BY p.product id;

-- S19: Xep hang khach hang theo doanh thu (RANK / DENSE RANK)
-- RANK bo so hang khi hoa, DENSE RANK thi khong — cot rank gap cho thay cho hai ham khac nhau.
SELECT c.customer id,
       c.name,
       ROUND(SUM(i.quantity * p.price), 0) AS revenue,
       RANK()       OVER (ORDER BY SUM(i.quantity * p.price) DESC) AS rank,
       DENSE RANK() OVER (ORDER BY SUM(i.quantity * p.price) DESC) AS dense rank
FROM customers c
JOIN orders o      ON o.customer id = c.customer id
JOIN order items i ON i.order id = o.order id
JOIN products p    ON p.product id = i.product id
GROUP BY c.customer id, c.name
ORDER BY revenue DESC
LIMIT 10;

-- S20: So sanh ke hoach thuc thi sau khi mo rong du lieu va tao index
-- Bang lon (500 don) nen chenh lech SCAN/SEARCH ro hon du lieu mau 6 don.
-- Cap EXPLAIN chi tiet (truoc/sau index, truoc/sau khi mo rong) nam trong sql/explain.sql.
CREATE INDEX IF NOT EXISTS idx_orders_customer ON orders(customer_id);
```

```
EXPLAIN QUERY PLAN
SELECT order id, order date
FROM orders
WHERE customer id = 7
ORDER BY order date DESC;

-- S21: Khac biet SQL Server (#table / ##table / tempdb) va SQLite TEMP TABLE
--   * SQL Server `#t`   — bang tam cuc bo, chi thay trong session da tao, tu xoa khi session
--     hoac procedure ket thuc; song trong tempdb (khong phai CSDL nguoi dung).
--   * SQL Server `##t`  — bang tam toan cuc, moi session deu thay, mat khi session cuoi cung
--     dang tham chieu no dong lai.
--   * SQL Server `tempdb` — CSDL he thong dung chung cho bang tam, bien bang, sort/hash spill;
--     co the truy van truc tiep `tempdb..#t`.
--   * SQLite `CREATE TEMP TABLE t` — bang tam nam trong CSDL `temp` (gan voi ket noi), chi
--     connection tao ra no thay, tu mat khi dong ket noi; KHONG co khai niem bang tam toan cuc
--     `##`.
--   * SQLite khong co tempdb tach roi: schema `temp` co the nam trong RAM hoac file tam, do
--     `PRAGMA temp_store` quyet dinh (0 = mac dinh, 1 = file, 2 = memory).
--   * Ca hai deu uu tien ten trong schema tam khi trung ten voi bang thuong (SQLite: bang
--     `temp` duoc tim truoc `main`).
-- Minh hoa bang TEMP TABLE (chi ton tai trong phien chay run sql.py nay):
CREATE TEMP TABLE revenue by month AS
SELECT strftime('%Y-%m', o.order date) AS month,
       SUM(i.quantity * p.price)       AS revenue
FROM orders o
JOIN order items i ON i.order id = o.order id
JOIN products p    ON p.product id = i.product id
GROUP BY month;

SELECT month, ROUND(revenue, 0) AS revenue
FROM revenue by month
ORDER BY revenue DESC
LIMIT 3;

DROP TABLE temp.revenue_by_month;
```

CHƯƠNG C. KẾT QUẢ ĐO VÀ ẢNH MINH CHỨNG

Bảy ảnh dưới đây được sinh tự động từ kết quả chạy thật của các kịch bản, không chỉnh sửa nội dung. Mỗi ảnh tương ứng một nhóm bằng chứng được nêu trong Chương 4.



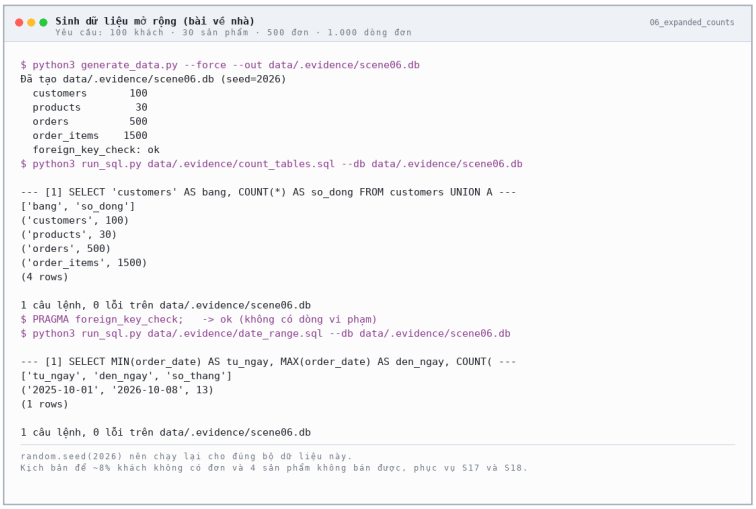
Hình C.1. Môi trường thực hành: phiên bản Python và SQLite



Hình C.2. Console SQLite: câu lệnh nhiều dòng, xử lý lỗi và thoát



Hình C.3. Số dòng bốn bảng sau khi nạp dữ liệu mẫu



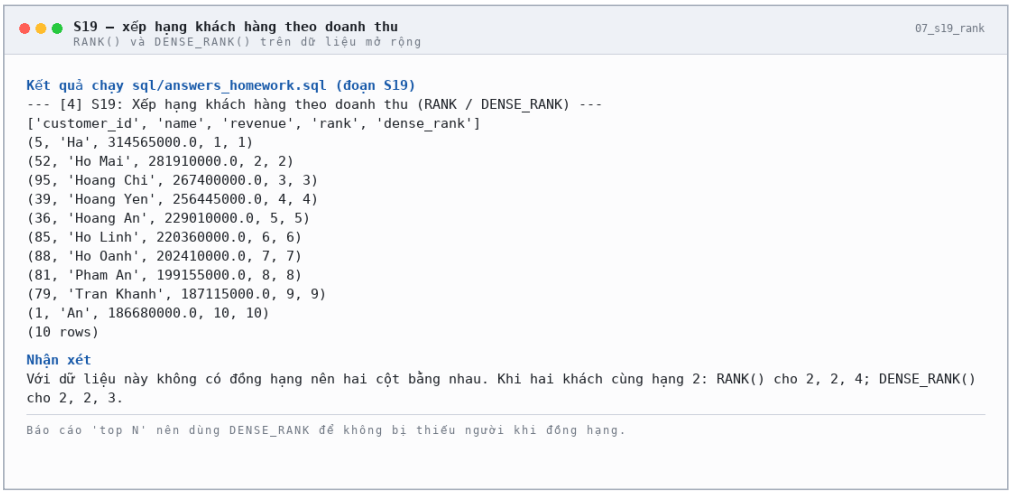
Hình C.4. Sinh dữ liệu mở rộng và kiểm tra khoá ngoại



Hình C.5. Kết quả S08 và S09



Hình C.6. Kế hoạch thực thi trước và sau khi tạo chỉ mục



Hình C.7. Kết quả xếp hạng khách hàng với RANK và DENSE_RANK